Project 8 (Bonus) — MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically

Type: Single agent (ReAct loop over dynamically discovered MCP tools) · **Stack:** Python, `mcp` (the official MCP Python SDK), OpenAI SDK (function calling), python-dotenv, pydantic, asyncio · **Level:** Intermediate–Advanced

Tagline: A ReAct agent whose tools aren't written into its own code — it connects to one or more MCP servers at startup, discovers what they offer, and calls them dynamically.

> Not part of the numbered Doc01-19 arc — a bonus/portfolio project built on 06_tools_function_calling's MCP sections and 07_ai_agents's ReAct loop, and the direct companion to Project 7. Read Doc06 and Doc07's Core Concepts first if you haven't.

Charter (what this project is)

Build a real single-agent ReAct loop (the same think→act→observe pattern from Doc07), except its tools aren't a hardcoded Python list — they're discovered at startup from one or more live MCP servers, and called through the MCP protocol instead of a direct function call. Nothing about the *loop* changes from Doc07; what changes is where the tools come from and how a tool call actually gets executed. This is the client that uses a server like Project 7's, or any other real MCP server.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4

The Story — what you're actually building

Every agent you've built so far has its tools written directly into its own code — a `@tool`-decorated function, sitting right there in `tools.py`, that the loop calls directly. That's fine when you own the tool. It stops being fine the moment the tool you want lives somewhere else — a teammate's notes server, a company-wide search index, a filesystem service someone else runs — and you don't want to reimplement it or copy-paste its logic into every agent that needs it.

MCP exists exactly for that handoff. A server stands somewhere (a subprocess on your machine, or a shared service) and exposes its tools over one standard protocol. Any client — including an agent you build — can connect, ask "what have you got?", and use whatever comes back, without either side knowing the other's internals ahead of time. That's the whole idea behind MCPBridge: an agent that never hardcodes a single tool. It connects, it asks, it uses whatever it's handed — the same ReAct loop from Doc07, pointed at a *discovered* tool list.

The Steps build this the same way Project 2 built up its Worker agent. Step 1 proves discovery alone works, with no agent yet. Step 2 wires that discovery into a real ReAct loop that picks and calls a tool it never knew about at write-time. Step 3 faces needing tools from *more than one* server at once, and the name-collision problem that creates. Step 4 makes the whole thing survive a server that isn't there, or drops mid-task — and ties it into one working demo.

> **Before you read further — think about it yourself:** if your agent's tool list isn't known until the program actually runs and connects to a server, how does that change the code you write to hand tools to the model? And if two different MCP servers both expose a tool called `search`, how should your agent decide which one a given tool call is meant to hit? Sit with both before you read the Steps.

Where This Fits

This isn't part of the main 5-project arc. It's a deep, hands-on look at **one real integration technology** — MCP — that's increasingly how production agents get their tools. It belongs in your portfolio as proof you can build the *consuming* side of a protocol, not just call functions you wrote yourself.

It's the direct companion to **Project 7** (an MCP *server*). Project 7 answers "how do I expose my own tools to any MCP client?" This project answers the other half: "how do I build a client agent that actually uses one?" Point MCPBridge at Project 7's server, at the example servers shipped here, or at a public reference server. Nothing here depends on Project 7 existing first — MCP is a standard protocol, and any conforming server works.

What to Read First (docs this project uses)

- 06_tools_function_calling — what MCP is, how servers and clients connect, MCP's three building blocks (Tools, Resources, Prompts), getting a tool's failure back to the model.
- 07_ai_agents — the ReAct loop (think→act→observe), the scratchpad, the hard step limit, common single-agent failures.
- 08b_async_prereq — optional; the `asyncio.run(main())` pattern this project reuses throughout.

Project Structure (the professional layout)

Same layout rule as every project since Project 1: an entry point (`main.py`) at the repo root, and all the real code in a `src/` package imported as `from src.x import ...`. No `pyproject.toml`,

no `pip install -e`. The example MCP servers live in `servers/` because they are standalone programs the client *launches as subprocesses*, not part of the client's own package.

The final layout (after Step 4)

```
<repo folder>/   (MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically/)
├── servers/
│   ├── files_server.py         example MCP server (files) for local testing
│   └── notes_server.py         example MCP server (notes) for local testing
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              create_client(), ask_model()
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py           get_logger() — logs to stderr
│   ├── __init__.py
│   ├── agent.py                run_agent() ReAct loop, AgentResult
│   ├── config.py               Config, load_config()
│   ├── exceptions.py           MissingConfigError, MaxIterationsExceeded, …
│   ├── server_pool.py          MCPServerPool: connect, discover, route
│   ├── servers_config.py       SERVERS = [{name, command, args}, …]
│   └── tool_bridge.py          mcp_tools_to_openai_schema(), extract_text()
├── tests/
│   ├── integration/
│   │   └── test_agent.py       real servers: all four cases
│   └── unit/
│       ├── test_config.py      no server, no network
│       └── test_tool_bridge.py
├── .env                        your real key — never committed
├── .env.example                the setting names, no real values
├── .gitignore
├── LICENSE
├── main.py                     the demo you run
├── README.md
└── requirements.txt
```

Why each folder exists

- `servers/` — the example MCP servers. They are not imported; the client launches them as subprocesses over stdio, exactly as it would any real server.
- `src/` — the client package: config, logging, the server pool, the schema bridge, the agent loop.
- `src/models/` — the one file that talks to OpenAI.
- `src/utils/` — small shared helpers (logging).

- `tests/` — unit tests (no network) and an integration test (real servers).

Why each file exists (what and why)

- `servers/notes_server.py`, `servers/files_server.py` — two tiny `MCPServer` servers so you can run and test the whole project with no external setup. Swap in Project 7's server or a public one anytime.
- `src/config.py` — settings loaded from `.env` into a checked `Config` (the Doc01 pattern), plus the timeout and iteration cap.
- `src/exceptions.py` — the project's named errors: `MissingConfigError`, `ServerUnreachableError`, `MaxIterationsExceeded`.
- `src/utils/logger.py` — one logging setup; logs to **stderr**, because an MCP client can carry protocol traffic on stdout.
- `src/server_pool.py` — `MCPServerPool`: connects to every configured server, discovers each one's tools, namespaces the names, and routes each call back to the right server. The heart of the project.
- `src/servers_config.py` — the list of servers to connect to, as plain data.
- `src/tool_bridge.py` — `mcp_tools_to_openai_schema()` turns discovered MCP tools into OpenAI function schemas; `extract_text()` pulls the text out of a `CallToolResult`.
- `src/models/llm.py` — the OpenAI client and the one `ask_model()` call the loop uses.
- `src/agent.py` — `run_agent()`, the hand-built ReAct loop, returning an `AgentResult`.
- `main.py` — the demo: connect, discover, run a task, print each step.

How the structure grows, step by step

- **Step 1** creates `config`, `exceptions`, `utils/logger`, `servers_config` (one server), `server_pool` (connect + discover + namespace), one example server, `main.py` (prints discovered tools), and `test_config`.
- **Step 2** adds `tool_bridge`, `models/llm`, `agent`, gives the pool `openai_tools()` + `call_tool()`, makes `main.py` run the agent, and adds `test_tool_bridge` + `test_agent`.
- **Step 3** adds the second example server and a second entry in `servers_config`; the pool's namespacing now earns its keep.
- **Step 4** hardens the pool (connect timeout + skip unreachable, catch mid-task failures) and `run_agent` (the `MaxIterationsExceeded` cap), and finishes the demo.

Setup (do this once, before Step 1)

```
cd MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

```
pip install mcp openai python-dotenv pydantic
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`, put your real key in `.env`):

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
MCP_CONNECT_TIMEOUT=10
MCP_MAX_ITERATIONS=6
```

`.gitignore`:

```
.env
.venv/
__pycache__/
*.pyc
```

Every folder under `src/` gets an empty `__init__.py` (`src/`, `src/models/`, `src/utils/`). This project is `asyncio`-based throughout, because the `mcp` SDK's client is async-only — but the pattern (`asyncio.run(main())` around an `async def`) is the same one Doc06's MCP client example already showed.

A Real Example (so this isn't just theory)

Use this scenario for every step — keep it this specific so you have real test cases:

Scenario: connect MCPBridge to two small servers: a **notes** server (`search_notes(query)`, `read_note(note_id)`) and a **filesystem** server (`list_files(directory)`, `read_file(path)`). The `servers/` folder ships both.

Test tasks: 1. `"What does my note about the Q3 roadmap say?"` — one server (notes): `search_notes`, then `read_note` with an id from the first result. 2. `"List the files in the project folder, then say if any look like a config file."` — one server (filesystem): `list_files`, then reasoning, no `read_file` needed. 3. `"Check my notes for the deploy process, and also list the files in the project folder."` — **both** servers in one run — proves namespacing and cross-server routing. 4. `"Look something up in my notes"` — asked with the notes server deliberately not started — the agent says clearly it can't reach those tools instead of hanging or crashing.

How testing works in this project

Three layers, one style. Because this project builds only on Doc06/07 (which use plain test scripts, not `pytest`), **every test is a plain OK/WRONG script**, one function per case, run with `python -m`:

- **`tests/unit/test_config.py`** — the settings code, no network. Uses a made-up setting name so it never touches your real `.env`.
- **`tests/unit/test_tool_bridge.py`** — the schema converter and text extractor, with tiny fake tool objects — no server needed.
- **`tests/integration/test_agent.py`** — launches the real example servers over a real MCP connection and checks discovery, chaining, cross-server routing, and the unreachable-server fallback.

Each case prints `OK` or `WRONG`. Every step below ends with a **Tests for this step** box: what it checks, why, when it should pass, and what a failure means.

Step 1 — Connect to One MCP Server and List Its Tools

What this step does: proves the smallest piece works before anything builds on it — a real client connecting to a real server over stdio and discovering what it offers. No agent, no model call — just connection and discovery.

Why this step matters: every later step depends on this connection. If discovery is broken, debugging Step 2's loop on top of it means debugging two things at once — the same reason Project 2 proved its chain alone first.

When you'll hit this for real: the first few minutes of connecting to *any* new MCP server — before you trust it enough to build on, you confirm it returns what you expect.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do: 1. Write `config.py`, `exceptions.py`, `utils/logger.py` (the Doc01 trio — logger to **stderr**). 2. Write one example server, `servers/notes_server.py`: a `MCPServer` with `search_notes` and `read_note` tools (Doc06's minimal shape). 3. Write `servers_config.py` with that one server, and `MCPServerPool` in `server_pool.py` with `connect_all()` — launch each server over stdio, `initialize()`, call `list_tools()`, and store each tool under a namespaced name `servername__toolname`. 4. Write `main.py` to connect and print every discovered tool's name and description. Run it once against the working server, once against a wrong command — confirm you get a clear failure, not a silent hang.

Your files after Step 1:

```
MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically/
├── servers/
│   └── notes_server.py
```

```
├── src/
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── server_pool.py          connect_all(), discovery, namespacing
│   └── servers_config.py       one server
├── tests/
│   └── unit/
│       └── test_config.py
├── .env.example
├── main.py                     prints discovered tools
└── requirements.txt
```

Tests for this step (`python -m tests.unit.test_config`): - **Check:** a missing setting and an empty setting both raise `MissingConfigError` ; a present key loads. - **Why:** a config bug should fail loudly at startup, not three calls deep. - **When:** should pass from Step 1 on. - **If not:** your `require_env` isn't treating an empty string as missing.

Step 2 — A Real ReAct Agent That Calls a Discovered Tool

What this step does: wires Step 1's discovered tools into an actual think→act→observe loop — the model picks a tool from what was discovered at runtime, and the agent executes it through the MCP client.

Why this step matters: this is the point of the whole project — an agent acting on tools it never had written into it. Step 1 proved *you* can see what a server offers; this proves the *model* can use it, with no `@tool` function anywhere.

What's new vs. Step 1: a model call, the loop, and a bridge that turns MCP's tool shape into OpenAI's function-calling shape. **What stays the same:** Step 1's pool and connection.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do: 1. Write `tool_bridge.py` : `mcp_tools_to_openai_schema(named_tools)` builds the `{"type": "function", "function": {name, description, parameters}}` dict OpenAI expects — an MCP tool's `input_schema` is already JSON Schema, so this is mostly a copy. `extract_text(call_result)` pulls text from the `CallToolResult.content` blocks. 2. Give `MCPServerPool` two methods: `openai_tools()` (the converted schemas) and `call_tool(namespaced_name, arguments)` (dispatch to the right session, return extracted text). 3. Write `models/llm.py` (`create_client` , `ask_model`) and `agent.py` : `run_agent(task, pool, client, max_iterations)` — a hand-built loop (no `create_agent`), sending the conversation + discovered schemas, running each tool call through `pool.call_tool` , appending a

`tool` message, repeating until no tool calls. Add a plain `max_iterations` cap now. 4. Make `main.py` run the agent on one of the real tasks. Test a one-call task and a two-chained-call task (`search_notes` → `read_note`), confirming the second call uses the first's result.

Your files after Step 2:

```
MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically/
├── servers/
│   └── notes_server.py
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── agent.py                run_agent(), AgentResult
│   ├── config.py
│   ├── exceptions.py
│   ├── server_pool.py          + openai_tools(), call_tool()
│   ├── servers_config.py
│   └── tool_bridge.py
├── tests/
│   ├── integration/
│   │   └── test_agent.py       discovery + chained-call cases
│   └── unit/
│       ├── test_config.py
│       └── test_tool_bridge.py
├── .env.example
├── main.py                     runs the agent
└── requirements.txt
```

Tests for this step: - **Check:** `test_tool_bridge` — the converter produces the right shape, a `None` description becomes `""` , `extract_text` joins blocks. `test_agent` cases 1–2 — tools are discovered and namespaced; the chained task calls `search_notes` then `read_note` . - **Why:** proves the bridge and the loop work on real discovered tools, not assumed shapes. - **When:** from Step 2 on. - **If not:** check you read `tool.input_schema` (snake_case in the current `mcp` SDK), not `inputSchema` .

Step 3 — Multiple MCP Servers at Once, With Tool Namespacing

What this step does: connects the agent to more than one server at once, and handles the real problem that creates — two servers can expose a tool with the same name, so a call has to know

which server it belongs to.

Why this step matters: a real agent rarely needs tools from only one source. Without namespacing, two same-named tools silently collide and the routing can't tell them apart.

What's new vs. Step 2: a second server in the config. **What stays the same:** the pool already namespaces (`servername__toolname`) and keeps a routing table from Step 1 — so the loop still just sees a flat list of names and calls one. The lesson of this step is *why* that namespacing was there.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do: 1. Write a second example server, `servers/files_server.py` (`list_files` , `read_file`), and add it to `servers_config.py` . 2. Confirm the pool namespaces every tool as `servername__toolname` . **Watch this gotcha:** OpenAI's function-name field allows only letters, digits, `_` and `-` — a `.` separator is invalid, so use `__` , and keep the routing dict (`{"notes__search_notes": ("notes", "search_notes")}`) mapping back to the real server + tool. 3. Run the "needs both servers" task. Confirm from the logs that each call routed to the server you expected — not just that the final answer looked right.

Your files after Step 3:

```
MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically/
├── servers/
│   ├── files_server.py         NEW — second server
│   └── notes_server.py
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── agent.py
│   ├── config.py
│   ├── exceptions.py
│   ├── server_pool.py
│   ├── servers_config.py       + the filesystem server
│   └── tool_bridge.py
├── tests/
│   ├── integration/
│   │   └── test_agent.py       + routes-to-both-servers case
│   └── unit/
│       ├── test_config.py
│       └── test_tool_bridge.py
├── .env.example
```

```
├── main.py                     runs a both-servers task
└── requirements.txt
```

Tests for this step: - **Check:** `test_agent` case 3 — the both-servers task uses `notes__search_notes` *and* `filesystem__list_files`. - **Why:** proves routing sends each namespaced call to the correct server. - **When:** from Step 3 on. - **If not:** check the routing dict maps the namespaced name back to the right `(server, tool)` pair.

Step 4 — A Fallback When a Server Is Unreachable, and a Full Working Demo

What this step does: makes the system survive a server that isn't running, never finishes starting, or drops mid-task — instead of hanging or crashing — and ties everything into one demo.

Why this step matters: every step so far assumed every server works. Production doesn't — a server can be down for a deploy, misconfigured, or slow. An agent that can't tell "one tool source is gone" from "everything is broken" isn't trustworthy.

What's new vs. Step 3: a connect timeout + `try/except` per server (a failed one is logged and skipped, not fatal), a `try/except` around each tool call (a mid-task drop becomes a clear observation), and `run_agent` raising `MaxIterationsExceeded` at the cap. **What stays the same:** every server that *does* connect behaves exactly as in Step 3.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do: 1. In `MCPServerPool.connect_all`, wrap each server's `initialize()` in `asyncio.wait_for(..., timeout=...)` inside a `try/except`. A server that times out or raises is logged (`"[notes] unreachable: …, continuing without it"`) and left out — the others still connect. **Enter the stdio/session context managers in the main task, never inside `wait_for`** (it runs its coroutine in a new task, and a context entered in one task can't be closed in another). 2. Wrap the actual `call_tool()` dispatch in its own `try/except` — a connected server can still drop mid-task. On failure, return a clear `Error: …` string as the observation instead of crashing the loop. Keep the `max_iterations` cap, now raised as `MaxIterationsExceeded`. 3. Write `main.py` as the final demo: connect to the configured servers (leave one misconfigured), run the both-servers task, and print which servers connected, each step, and the answer. 4. Test it: full demo with everything up, then again with one server never started — confirm the agent finishes on what's left and says it couldn't reach the missing one.

Your files after Step 4 (final): the full layout shown above.

Tests for this step: - **Check:** `test_agent` case 4 — a broken server is skipped (`"broken" not in pool.sessions`), the good one connects, and the agent still produces an answer. - **Why:** proves one unreachable server degrades the agent instead of crashing it. - **When:** from Step 4 on. - **If not:**

confirm the `try/except` in `connect_all` only skips the failing server, and that you didn't wrap the context-manager entry in `wait_for`.

Final Deliverable: a single ReAct agent that connects to one or more real MCP servers, discovers their tools at runtime, routes namespaced tool calls to the right server, and keeps working on whatever tools remain when a server is unreachable.

Run, Test and Configure the Final Project

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity
`MCP_CONNECT_TIMEOUT`	no	`10`	Seconds to wait for a server to start
`MCP_MAX_ITERATIONS`	no	`6`	Hard cap on ReAct steps

Usage

```
python main.py
```

Running the Tests

```
# unit — no server, no network
python -m tests.unit.test_config
python -m tests.unit.test_tool_bridge

# integration — launches the example servers over a real MCP connection
python -m tests.integration.test_agent
```

Checklist Before You Call This Done

- [ ] The tool list comes entirely from `list_tools()` at runtime — no tool written into the agent's own code
- [ ] The ReAct loop is hand-built (think→act→observe), not `create_agent` / `AgentExecutor`
- [ ] At least 2 servers connected at once, names namespaced and routed back correctly

- [ ] A test where the agent needs tools from both servers, and the logs show each call hit the right server
- [ ] A test where one server is unreachable — the agent logs it, keeps running, and doesn't hang or crash
- [ ] A tool call that fails mid-task is fed back to the model as a clear observation, not an unhandled exception
- [ ] You can explain, for a namespaced call, exactly which code decided which server it goes to

Publish It to GitHub

Give the repo folder the professional name `MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically`, add the `README.md`, commit the `.env.example` (never `.env`), and push. It's a strong portfolio piece: it shows you can build the consumer side of a real protocol, with resilience, not just call functions you wrote yourself.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**